

Demo Runbook 4

Hyperparameter Optimization • Model Packaging • API Development for Predictions

Copy-paste-ready commands, expected outputs, and talking points for running each lab live as a demo.

How to use this	Run the Setup block once before the audience arrives. During the demo, execute each numbered Step live, pausing on the 'Expected output' box so the audience sees the result before you move on.
Total demo time	Approx. 30-40 minutes for all 3 labs

LAB 1

Hyperparameter Optimization

Replacing manual guess-and-check tuning with an automated search, live

Demo Goal: Show the difference between hand-picking hyperparameters and letting Optuna search the space automatically, converging on a better result in view of the audience.

Setup (run once before the demo)

```
pip install optuna scikit-learn
mkdir hpo_demo && cd hpo_demo
```

STEP 1

Start with the 'manual guessing' approach

```
cat > manual_tune.py << 'EOF'
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier

X, y = load_wine(return_X_y=True)

# Guess 1
score1 = cross_val_score(RandomForestClassifier(n_estimators=50, max_depth=3, random_state=42),
    X, y, cv=5).mean()
print('Guess 1 (n_estimators=50, max_depth=3):', score1)

# Guess 2 – try something bigger
score2 = cross_val_score(RandomForestClassifier(n_estimators=300, max_depth=20, random_state=42),
    X, y, cv=5).mean()
print('Guess 2 (n_estimators=300, max_depth=20):', score2)
EOF
python manual_tune.py
```

Expected output / what you'll see:

```
Guess 1 (n_estimators=50, max_depth=3): 0.9382...
Guess 2 (n_estimators=300, max_depth=20): 0.9719...
```

Say to audience: "This is how most tuning happens in practice — try a couple of combinations by hand, pick whichever looks best, and stop. We have no idea if there's a better combination we didn't happen to try."

STEP 2

Now let Optuna search automatically

```
cat > auto_tune.py << 'EOF'
import optuna
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

X, y = load_wine(return_X_y=True)

def objective(trial):
    n_estimators = trial.suggest_int('n_estimators', 10, 300)
    max_depth = trial.suggest_int('max_depth', 2, 20)
    model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth, random_state=42)
    return cross_val_score(model, X, y, cv=5).mean()

EOF
```

```
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20, show_progress_bar=False)

print('Best score:', study.best_value)
print('Best params:', study.best_params)
EOF
python auto_tune.py
```

Expected output / what you'll see:

```
[I 2026-...] Trial 0 finished with value: 0.933...
... (20 trials print) ...
Best score: 0.9832...
Best params: {'n_estimators': 187, 'max_depth': 11}
```

Say to audience: *"In 20 automated trials, Optuna found a combination — `n_estimators` around 187, `max_depth` around 11 — that neither manual guess would have landed on, and it beat both of our hand-picked attempts."*

STEP 3

Visualize the search (optional, if plotly is installed)

```
pip install plotly
python -c "
import optuna
# re-run study or reuse the in-memory 'study' object in a notebook/REPL
# optuna.visualization.plot_optimization_history(study).show()
"
```

Expected output / what you'll see:

An interactive chart showing each trial's score converging upward over 20 trials — a strong visual for a live audience.

Say to audience: *"If you're in a notebook, this chart makes the search process itself visible — it's a great close for this segment."*

Demo Wrap-up

Mention this connects directly back to the Configuration Management demo earlier in the series — under the hood, Optuna is just generating and testing many config combinations automatically.

LAB 2

Model Packaging

Packaging a trained model as a self-contained, portable MLflow model

Demo Goal: Show a model being packaged with everything needed to run it elsewhere — code, dependencies, and the model itself — and then run it as a standalone local REST server with zero custom serving code.

Setup (run once before the demo)

```
pip install mlflow scikit-learn
mkdir model_packaging_demo && cd model_packaging_demo
```

STEP 1

Train and package the model with MLflow's standard format

```
cat > package_model.py << 'EOF'
import mlflow, mlflow.sklearn
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier

X, y = load_wine(return_X_y=True)
model = RandomForestClassifier(n_estimators=100).fit(X, y)

mlflow.sklearn.save_model(model, 'packaged_model', input_example=X[:2])
print('Model packaged to ./packaged_model')
EOF
python package_model.py
```

Expected output / what you'll see:

```
Model packaged to ./packaged_model
```

STEP 2

Inspect what packaging actually produced

```
find packaged_model -type f
cat packaged_model/MLmodel
```

Expected output / what you'll see:

```
packaged_model/MLmodel
packaged_model/model.pkl
packaged_model/conda.yaml
packaged_model/requirements.txt
packaged_model/python_env.yaml
```

(MLmodel file shows the model's flavor, signature, and input example — a self-describing manifest.)

Say to audience: "Packaging isn't just 'save the pickle file.' It's the model plus a manifest describing its inputs and outputs, plus the exact dependency versions needed to run it anywhere — that's what 'portable' actually means."

STEP 3

Serve it as a REST API with zero custom serving code

```
mlflow models serve -m packaged_model -p 5001 --no-conda
# (leave this running in one terminal)
```

Expected output / what you'll see:

Serving on http://127.0.0.1:5001

Say to audience: *"This is the payoff: nobody wrote a single line of Flask or FastAPI code, yet we have a working prediction API — purely from the packaging format."*

STEP 4

Call the served model from a second terminal

```
curl -X POST http://127.0.0.1:5001/invocations \  
-H 'Content-Type: application/json' \  
-d '{"dataframe_split": {"columns": [0,1,2,3,4,5,6,7,8,9,10,11,12],  
    "data": [[13.2,1.78,2.14,11.2,100,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050]]}'
```

Expected output / what you'll see:

```
{"predictions": [0]}
```

Demo Wrap-up

Stop the 'mlflow models serve' process before moving to Lab 3, since it holds port 5001.

LAB 3

API Development for Predictions

Building a custom FastAPI prediction service with input validation, live

Demo Goal: Show a hand-built prediction API with request validation, so the audience sees both the auto-generated approach (Lab 2) and the fully custom-code approach for comparison.

Setup (run once before the demo)

```
pip install fastapi uvicorn scikit-learn joblib pydantic
mkdir api_demo && cd api_demo
```

STEP 1

Train and save a model to serve

```
python3 -c "
import joblib
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
X, y = load_wine(return_X_y=True)
model = RandomForestClassifier(n_estimators=100).fit(X, y)
joblib.dump(model, 'model.joblib')
print('Saved model.joblib')
"
```

Expected output / what you'll see:

```
Saved model.joblib
```

STEP 2

Write the FastAPI app with input validation

```
cat > main.py << 'EOF'
import joblib
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, conlist

app = FastAPI(title='Wine Classifier API')
model = joblib.load('model.joblib')

class PredictRequest(BaseModel):
    features: conlist(float, min_length=13, max_length=13)

@app.get('/health')
def health():
    return {'status': 'ok'}

@app.post('/predict')
def predict(req: PredictRequest):
    try:
        prediction = model.predict([req.features])[0]
        return {'prediction': int(prediction)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

EOF
```

STEP 3

Start the API server

```
uvicorn main:app --reload --port 8000
# (leave this running in one terminal)
```

Expected output / what you'll see:

```
Uvicorn running on http://127.0.0.1:8000
```

STEP 4

Show the auto-generated interactive docs

```
# open in browser: http://127.0.0.1:8000/docs
```

Expected output / what you'll see:

A live Swagger UI listing `/health` and `/predict`, with a 'Try it out' button – FastAPI generates this automatically from the Pydantic model.

Say to audience: *"This documentation wasn't written by hand — it was generated directly from the same type hints that validate incoming requests. One source of truth for both validation and documentation."*

STEP 5

Call `/predict` with a valid request

```
curl -X POST http://127.0.0.1:8000/predict \
-H 'Content-Type: application/json' \
-d '{"features": [13.2, 1.78, 2.14, 11.2, 100, 2.65, 2.76, 0.26, 1.28, 4.38, 1.05, 3.4, 1050]}'
```

Expected output / what you'll see:

```
{"prediction":0}
```

STEP 6

Now send an intentionally invalid request live

Send only 3 features instead of the required 13, to show validation rejecting it before it ever reaches the model.

```
curl -X POST http://127.0.0.1:8000/predict \
-H 'Content-Type: application/json' \
-d '{"features": [1.0, 2.0, 3.0]}'
```

Expected output / what you'll see:

```
{"detail":[{"type":"too_short", ... "msg":"List should have at least 13 items after validation, not 3" ...}]}
```

Say to audience: *"This is the point: the malformed request never reached `model.predict()` at all — it was rejected at the API boundary with a clear, structured error. That's what stands between 'user sends bad input' and 'the service crashes or silently mispredicts.'"*

Demo Wrap-up

Good closing line for the full four-PDF demo series: 'We've gone from a manual notebook script, to tracked experiments, to a governed registry, to a validated, documented, callable API — that's the MLOps lifecycle end to end.'